

Semi-Global Block Matching Acceleration

Ishan Gupta, Asavri Kaur, Vasilis Pteris, Alexandra Ukrainskaya

Project Overview



Motivation:

Accelerate and improve the Semi-Global Block Matching (SGBM) algorithm explicitly for aerial drone land surveying.

Starting Point:

Vitis Vision Library includes an optimized SGBM implementation. We built on it.

Methodology:

Identified 4 functions that are key to the SGBM algorithm to distribute between each member of our team.

- Census Transform
- Matching Cost Computation
- Cost Aggregation
- Disparity Selection

Objectives & Outcomes

Initial Objectives:

- Achieve a 10x hardware speedup compared to software benchmark.
- Keep resource utilization < 70%
- Verify correctness of output disparity maps
- **Bonus:** Achieve throughput above 24fps for real time SGBM

Outcomes:

- Achieved **~797x** speedup against software and **~0.73x** against original Vitis HLS
- Resources (%) - BRAM: 28, DSP: 41, FF: 18, LUT: 98, URAM: 87
- Compared disparity maps against ground truths for the Drone dataset and using pixel by pixel comparison scripts
- Maximum throughput **~55fps**

4 Algorithm Overview

Create a local descriptor



Search horizontally along the corresponding row



Calculate a matching cost for each candidate disparity



Consider disparity consistency with neighboring pixels.



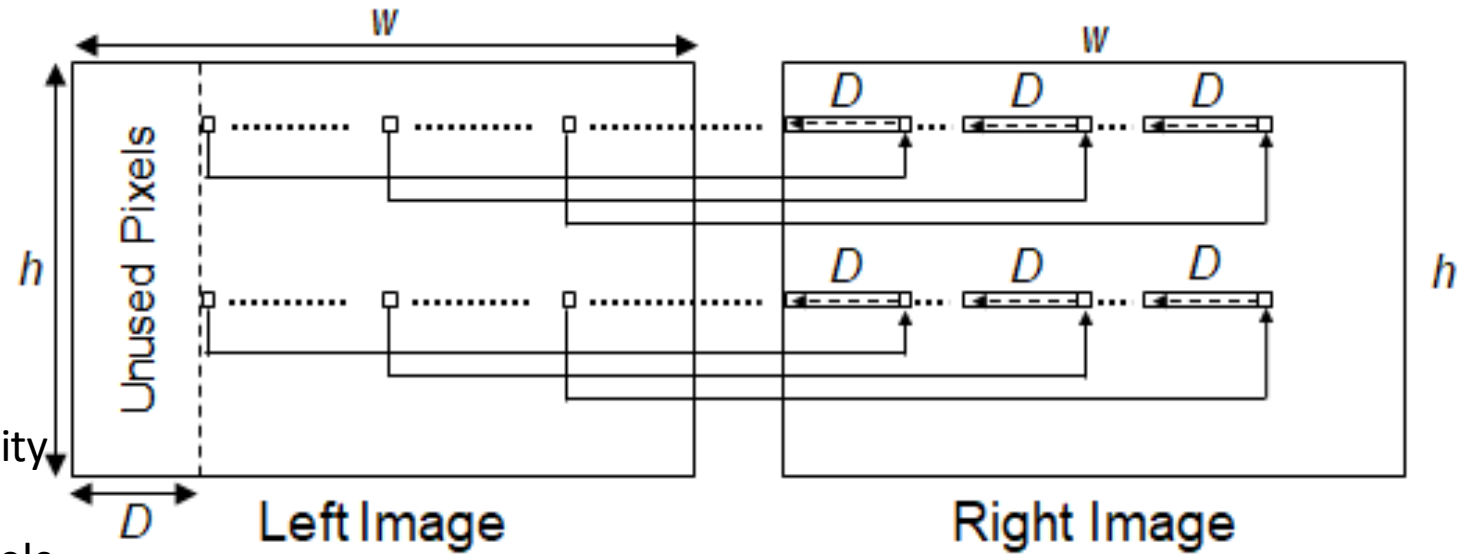
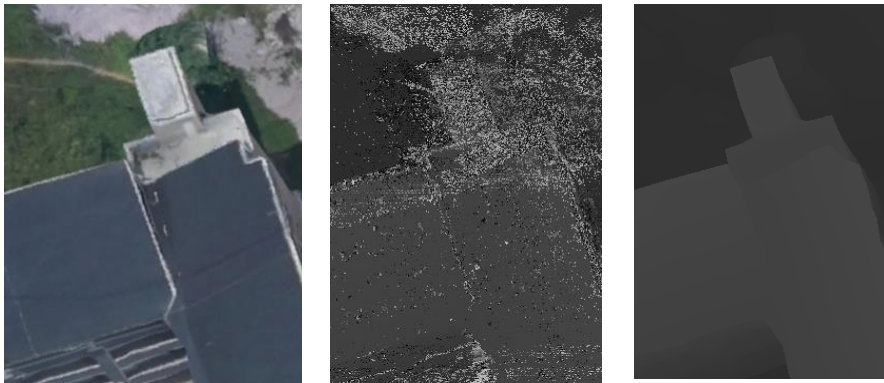
Update the costs of each candidate.



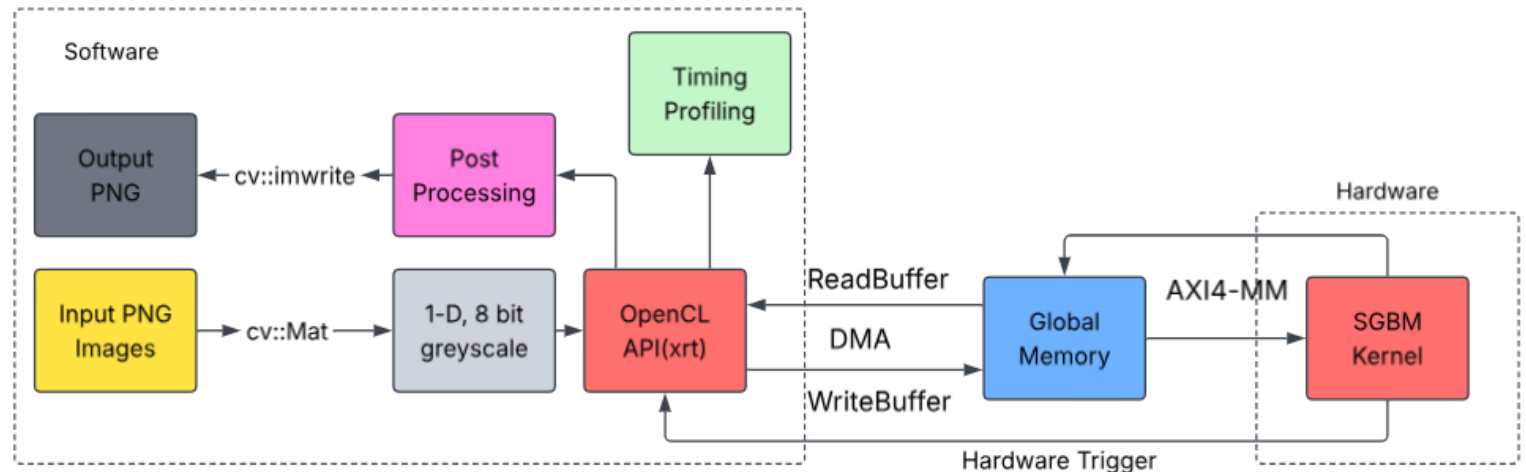
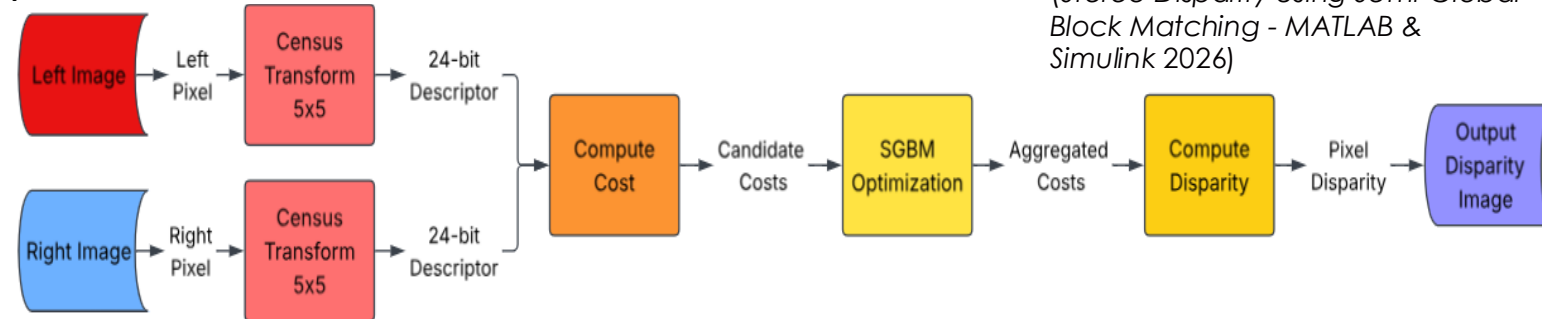
Select the candidate with the lowest total cost



Output its horizontal shift as the disparity



(Stereo Disparity Using Semi-Global Block Matching - MATLAB & Simulink 2026)



One tool per dataset

Middlebury ships an official evaluation SDK. Our drone captures don't, so we wrote our own.

Middlebury dataset

Middlebury evaluation SDK

- Published ground truth, official metric definitions
- Numbers directly comparable to other work



Drone dataset

Our own script

- No evaluation tool exists for our captures
- Reads ground truth and HLS output as plain PNGs



Same three metrics either way: **average error** • **bad pixel percent**

Baseline Performance Profiling

Stage	Latency (Clock Cycles)	BRAM(%)	DSP(%))	FF(%)	LUT(%)
Array2xfMat (Matrix <-> Stream) and Mat2Array	295000	0	~0	~0	1
xFCensusTransformKernel	303000	1	0	~0	2
xFSGBMoptimization	1329000	67	20	2	32
xFSGBMcomputecost	589800	0	~0	~0	1
xFSGBMcomputedisparity	589800	0	~0	~0	1

Clock = 10ns
(5ns  Timing not met)

Total Latency (cycles)
= 1329000

Execution time
= 13.29ms

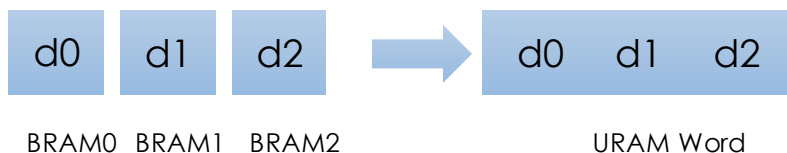
**Estimated clock
frequency**
= $1/(7.3\text{ns})$
= 136.986 MHz

**Measured Latency
(OpenCL)** = 13.254ms

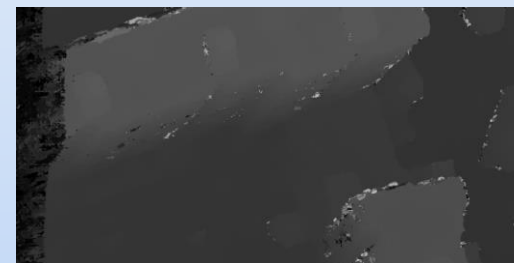
Total bad pixels =
56.82%

Algorithmic Improvements

1. Using URAMs for Lr buffer



2. Increasing Number of Disparity
3. Fusing Census cost with Absolute Difference
4. Compute disparity at sub-pixel level
5. Post-Processing



Results

Approach	Latency	BRAM (%)	DSP (%)	FF (%)	LUT (%)	URAM (%)	Total bad pixels (%)
Software	14515.1ms	NA	NA	NA	NA	NA	56.82
Vitis Library	13.25ms	72	20	9	60	0	56.82
Proposed (Hardware)	11.87ms	5	20	8	47	31	56.32
Proposed (Accuracy)	18.19ms	28	41	18	98	87	31.08

Transfer Direction	Time (ms)
Host -> Board (left and right images)	0.65
Board -> Host (output image)	0.43

Clock = 10ns

Total Latency (cycles)

= 1819000

Execution time = 18.19ms

Estimated clock frequency

= 1/(7.3ns)

= 136.986 MHz

Measured Latency (OpenCL) = 18.19ms

Total bad pixels = 31.08%

Discussion

What we achieved

- Increased accuracy without significant slow-down
- Decreased BRAM consumption

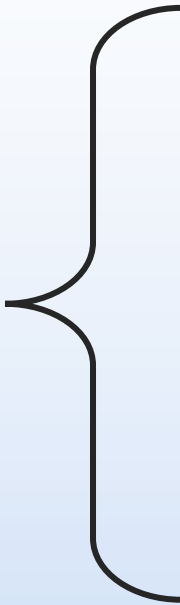


What we could do

- Post-processing
- Decrease resource usage in non-bottleneck functions
- Add real-time cameras
- Solve the noise problem
- Try faster clock



Thank You



Ishan Gupta

Asavri Kaur

Vasilis Pteris

Alexandra Ukrainskaya